

SISTEMAS DE GESTÃO DE BASES DE DADOS

Ficheiros e Índices em SGBD

Trabalho Académico — Curso de Engenharia Informática

Disciplina:

Sistemas de Gestão
de Bases de Dados

Docente:

Prof. Dr. [Moyo
Kanivengidio]

Estudantes:

[Bengui Bena Pedro | Cardoso Finiquissa | Egas
ManuelRibeiro | Fernando Lourenço Augusto Jorge|
Manuel António Tolentino e Teodora Miguel Domingos
]

Data:

Abril de 2026

1. Introdução

A gestão eficiente de dados é um dos pilares fundamentais dos Sistemas de Gestão de Bases de Dados (SGBD). Neste contexto, a organização dos ficheiros de dados e a criação de estruturas de índices desempenham um papel crítico no desempenho, escalabilidade e fiabilidade dos sistemas de informação modernos.

Este trabalho tem como objectivo explorar em profundidade os conceitos de ficheiros e índices em SGBD, abordando as diferentes estratégias de organização, os tipos de índices disponíveis, os seus algoritmos subjacentes e as implicações práticas na optimização de consultas (queries).

A compreensão destes mecanismos é essencial para qualquer profissional de tecnologia da informação que trabalhe com sistemas de bases de dados, pois permite tomar decisões fundamentadas sobre arquitectura, modelação e optimização de sistemas de armazenamento de dados.

2. Ficheiros em SGBD

2.1 Conceito e Estrutura

Em SGBD, um ficheiro é a unidade lógica de armazenamento persistente de dados em disco. Ao contrário do que ocorre em sistemas de ficheiros convencionais, os SGBD gerem internamente a organização física dos dados, abstraindo os utilizadores e aplicações da complexidade do armazenamento em disco.

A unidade mínima de transferência entre disco e memória designa-se bloco (ou página). O tamanho típico de um bloco varia entre 4 KB e 64 KB, dependendo da configuração do sistema operativo e do SGBD. Um ficheiro é constituído por um conjunto de blocos distribuídos em disco.

Conceito-chave: Bloco de Disco

O bloco de disco é a unidade fundamental de I/O em SGBD. Toda a leitura e escrita em disco é efectuada ao nível do bloco, mesmo que apenas um único registo seja necessário. Este facto tem implicações directas no desempenho das operações de leitura e escrita, tornando a organização dos ficheiros um factor crítico na optimização do sistema.

2.2 Métodos de Organização de Ficheiros

A organização dos ficheiros determina a forma como os registos são fisicamente armazenados e acedidos. Existem três métodos principais:

2.2.1 Organização em Heap (Não Ordenada)

Na organização em heap, os registos são inseridos sequencialmente sem qualquer ordenação. É a forma mais simples de organização e apresenta as seguintes características:

- ▶ Inserção eficiente: $O(1)$ — os novos registos são adicionados ao final do ficheiro.
- ▶ Pesquisa ineficiente: $O(n)$ — obriga a uma pesquisa linear em todos os blocos.
- ▶ Eliminação: marca o registo como eliminado, sem reorganização imediata.
- ▶ Adequado para: cargas de trabalho com predominância de inserções e leituras totais (full table scans).

2.2.2 Organização Sequencial (Ordenada)

Os registos são armazenados ordenados segundo um ou mais atributos chave. Esta organização permite pesquisas mais eficientes em determinados contextos:

- ▶ Pesquisa binária: $O(\log n)$ blocos para localizar um registo por chave de ordenação.
- ▶ Inserção custosa: pode exigir reorganização para manter a ordem.
- ▶ Eficiente para: consultas de intervalo (range queries) e operações de junção (merge join).

2.2.3 Organização por Dispersão (Hash)

Neste método, uma função de dispersão (hash function) é aplicada ao valor da chave para determinar o endereço físico do bloco onde o registo deve ser armazenado. Características principais:

- ▶ Pesquisa por igualdade: $O(1)$ — acesso directo ao bloco correspondente.
- ▶ Ineficiente para consultas de intervalo e ordenações.
- ▶ Colisões: tratadas por encadeamento (chaining) ou endereçamento aberto.
- ▶ Aplicável em: dicionários de dados, tabelas de hash joins e caches de SGBD.

Organização	Inserção	Pesquisa Exacta	Pesquisa Intervalo	Eliminação
Heap	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Sequencial	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Hash	$O(1)$	$O(1)$	$O(n)$	$O(1)$

Tabela 1 — Comparação da complexidade dos métodos de organização de ficheiros

3. Índices em SGBD

3.1 Definição e Importância

Um índice é uma estrutura de dados auxiliar que permite acelerar a recuperação de registos numa base de dados, análoga ao índice remissivo de um livro. Os índices são construídos sobre um ou mais atributos de uma tabela e armazenam pares (valor_chave, ponteiro_para_registro), permitindo localizar rapidamente os registos sem percorrer toda a tabela.

A importância dos índices cresce exponencialmente com o volume de dados. Numa tabela com milhões de registos, a diferença entre uma pesquisa com índice ($O(\log n)$) e sem índice ($O(n)$) pode representar a diferença entre milissegundos e minutos de tempo de resposta.

Tradeoff Fundamental dos Índices

Os índices melhoram significativamente o desempenho das operações de leitura (SELECT), mas introduzem overhead nas operações de escrita (INSERT, UPDATE, DELETE), pois cada modificação nos dados exige também a actualização dos índices correspondentes. A decisão de criar um índice deve sempre ponderar este equilíbrio entre velocidade de leitura e custo de manutenção.

3.2 Classificação dos Índices

3.2.1 Índices Primários e Secundários

O índice primário é construído sobre o atributo de ordenação do ficheiro de dados (geralmente a chave primária). Existe apenas um índice primário por ficheiro. O índice secundário, por sua vez, é construído sobre qualquer outro atributo que não o de ordenação, permitindo pesquisas eficientes por múltiplos critérios.

3.2.2 Índices Densos e Esparsos

Um índice denso contém uma entrada para cada registo do ficheiro de dados. Permite localizar qualquer registo directamente, mas ocupa mais espaço. O índice esperso contém apenas uma entrada por bloco de dados, tornando-o mais compacto, mas exigindo uma pesquisa interna no bloco após a localização.

3.2.3 Índices Agrupados e Não-Agrupados

Num índice agrupado (clustered index), a ordem física dos registos no disco coincide com a ordem lógica do índice. Apenas pode existir um índice agrupado por tabela. Os índices não-agrupados (non-clustered) não alteram a ordem física dos dados, podendo existir múltiplos por tabela.

4. Estruturas de Índice: Árvores B+

4.1 Motivação e História

A Árvore B+ é a estrutura de dados de índice mais amplamente utilizada em SGBD relacionais modernos, incluindo sistemas como PostgreSQL, MySQL (InnoDB), Oracle e SQL Server. Foi proposta por Rudolf Bayer e Edward McCreight em 1972 como extensão das Árvores B, com a modificação fundamental de armazenar todos os dados nas folhas.

4.2 Estrutura da Árvore B+

Uma Árvore B+ de ordem n é uma árvore de pesquisa balanceada com as seguintes propriedades fundamentais:

- ▶ Cada nó interno contém entre $\lceil n/2 \rceil$ e n chaves de pesquisa.
- ▶ Cada nó folha contém entre $\lceil (n-1)/2 \rceil$ e $n-1$ pares (chave, ponteiro para registo).
- ▶ Todos os nós folha estão ao mesmo nível (a árvore é perfeitamente balanceada).
- ▶ Os nós folha estão encadeados sequencialmente, facilitando consultas de intervalo.
- ▶ Os nós internos actuam como guias para a navegação eficiente na árvore.

4.3 Operações na Árvore B+

4.3.1 Pesquisa

A pesquisa numa Árvore B+ inicia-se na raiz e desce recursivamente pelos nós internos até atingir um nó folha. Em cada nó interno, compara-se o valor pesquisado com as chaves armazenadas para determinar o ramo a seguir. A complexidade temporal é $O(\log_n N)$, onde N é o número total de registos.

4.3.2 Inserção

A inserção localiza primeiro o nó folha apropriado. Se houver espaço, o par é inserido de forma ordenada. Caso contrário, o nó é dividido (split) em dois, propagando uma chave para o nó pai. Este processo pode propagar-se até à raiz, aumentando a altura da árvore em um nível quando necessário.

4.3.3 Eliminação

A eliminação remove o par do nó folha. Se o nó ficar com menos do que o mínimo de chaves, pode ser redistribuído com um nó irmão ou fundido (merge). A fusão pode propagar-se para os nós internos, podendo reduzir a altura da árvore.

Vantagem da Árvore B+ em SGBD

A principal vantagem da Árvore B+ sobre outras estruturas de índice reside no facto de os dados reais estarem sempre nas folhas, e as folhas estarem encadeadas. Isto permite que consultas de intervalo do tipo `WHERE salary BETWEEN 1000 AND 5000` sejam processadas de forma extremamente eficiente: localiza-se o primeiro valor por pesquisa na árvore e percorrem-se as folhas sequencialmente até ao último valor.

5. Índices de Dispersão (Hash)

5.1 Hash Estático

No hash estático, o número de buckets (compartimentos) é fixo e determinado pela função de dispersão $h(k) = k \bmod M$, onde M é o número de buckets. Esta abordagem é simples mas apresenta problemas de escalabilidade: se a base de dados crescer significativamente, o número de colisões aumenta e o desempenho degrada-se.

5.2 Hash Dinâmico (Extensível)

O hash extensível resolve as limitações do hash estático através de um mecanismo de duplicação dinâmica dos buckets. Utiliza-se um directório global que mapeia prefixos de bits para buckets. Quando um bucket fica cheio, divide-se apenas esse bucket, actualizando os ponteiros no directório. Desta forma, a tabela cresce de forma incremental e controlada.

5.3 Comparação: B+ vs. Hash

Critério	Árvore B+	Índice Hash
Pesquisa por igualdade	$O(\log n)$	$O(1)$ esperado
Pesquisa de intervalo	Muito eficiente	Ineficiente
Inserção	$O(\log n)$	$O(1)$ esperado
Suporte a ORDER BY	Sim (nativo)	Não
Utilização em SGBD	Universal	Selectiva

Tabela 2 — Comparação entre Árvore B+ e Índice Hash

6. Optimização de Consultas com Índices

6.1 Selectividade do Índice

A selectividade é uma métrica que indica a proporção de registos que uma condição de pesquisa irá retornar. Um índice de alta selectividade (ex: chave primária) retorna poucos registos por valor, sendo extremamente eficiente. Um índice de baixa selectividade (ex: coluna booleana) oferece pouco benefício, pois o optimizador pode optar por um full table scan.

6.2 Índices Compostos

Um índice composto é criado sobre múltiplas colunas simultaneamente. A ordem das colunas no índice é crucial: o índice (A, B, C) pode ser utilizado eficientemente em consultas que filtram por A, por A e B, ou por A, B e C, mas não em consultas que filtram apenas por B ou C. Esta propriedade designa-se de regra do prefixo mais à esquerda (leftmost prefix rule).

6.3 Índices de Cobertura

Um índice de cobertura (covering index) contém todas as colunas necessárias para satisfazer uma consulta, sem necessidade de aceder à tabela principal. O SGBD pode resolver a consulta inteiramente a partir do índice, eliminando o acesso ao disco para os dados. Este padrão é amplamente utilizado em sistemas OLAP e data warehouses para maximizar o desempenho.

Boas Práticas na Criação de Índices

Devem ser criados índices sobre: (1) colunas frequentemente utilizadas em cláusulas WHERE, JOIN e ORDER BY; (2) chaves estrangeiras de relações frequentemente juntas; (3) colunas de alta selectividade. Devem ser evitados índices sobre: tabelas muito pequenas (full scan pode ser mais rápido); colunas com baixa cardinalidade; tabelas com operações de escrita muito frequentes. O número excessivo de índices degrada o desempenho das operações DML (Data Manipulation Language).

7. Casos Práticos e Exemplos

7.1 Criação de Índices em SQL

A criação de índices em SQL é realizada através do comando CREATE INDEX. Os seguintes exemplos ilustram os cenários mais comuns:

```
-- Índice simples sobre chave estrangeira
CREATE INDEX idx_pedidos_cliente
ON pedidos (cliente_id);

-- Índice composto para consultas frequentes
CREATE INDEX idx_vendas_data_produto
ON vendas (data_venda, produto_id);

-- Índice único sobre coluna com restrição de unicidade
CREATE UNIQUE INDEX idx_utilizador_email
ON utilizadores (email);
```

Exemplo 1 — Criação de índices em SQL padrão

7.2 Análise de Planos de Execução

A maioria dos SGBD modernos disponibiliza ferramentas para análise do plano de execução de uma consulta, permitindo verificar se os índices estão a ser utilizados pelo optimizador. No PostgreSQL, utiliza-se o comando EXPLAIN ANALYZE; no MySQL, o comando EXPLAIN; no Oracle, o EXPLAIN PLAN FOR. Estes planos indicam se a operação utiliza Index Scan, Index Only Scan, Bitmap Index Scan ou Sequential Scan (full table scan), cada um com implicações diferentes no desempenho.

8. Estruturas de Índice Avançadas

8.1 Índices de Texto Completo (Full-Text)

Os índices de texto completo permitem pesquisas eficientes sobre campos de texto de grande dimensão, utilizando técnicas de análise linguística como tokenização, stemming e eliminação de stop words. São utilizados em aplicações de pesquisa documental e motores de busca internos.

8.2 Índices Espaciais (R-Trees)

Os índices R-Tree são utilizados para indexar dados geoespaciais, como coordenadas geográficas, polígonos e geometrias. Permitem responder eficientemente a consultas de proximidade espacial (ex: encontrar todos os pontos num raio de 5 km de uma localização). São amplamente utilizados em sistemas de informação geográfica (SIG) e aplicações de geolocalização.

8.3 Índices de Bitmap

Os índices de bitmap armazenam, para cada valor distinto de uma coluna, um vector de bits indicando quais os registos que possuem esse valor. São extremamente eficientes para colunas de baixa cardinalidade (poucos valores distintos) em ambientes OLAP, onde múltiplos índices de bitmap podem ser combinados através de operações lógicas (AND, OR, NOT) antes de aceder aos dados.

9. Conclusão

Os ficheiros e índices constituem a fundação do armazenamento e acesso eficiente de dados em SGBD. A escolha adequada da organização de ficheiros e das estruturas de índice tem um impacto directo e mensurável no desempenho dos sistemas de informação.

A Árvore B+ mantém-se como a estrutura de índice dominante nos SGBD relacionais, pela sua versatilidade em suportar tanto pesquisas exactas como consultas de intervalo. Os índices de dispersão (hash) complementam esta estrutura em cenários onde apenas pesquisas por igualdade são necessárias, oferecendo complexidade $O(1)$ esperada.

O profissional de bases de dados deve dominar estes conceitos para tomar decisões informadas na arquitectura de sistemas de informação, equilibrando o benefício das leituras aceleradas com o custo de manutenção dos índices nas operações de escrita.

Em suma, o conhecimento profundo das estruturas de armazenamento e indexação é indispensável para o desenvolvimento de sistemas de bases de dados robustos, escaláveis e de alto desempenho, que são a espinha dorsal de qualquer aplicação empresarial moderna.

Referências Bibliográficas

RAMAKRISHNAN, R.; GEHRKE, J. — *Database Management Systems*. 3.^a ed. McGraw-Hill, 2003.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. — *Database System Concepts*. 7.^a ed. McGraw-Hill, 2019.

ELMASRI, R.; NAVATHE, S. B. — *Fundamentals of Database Systems*. 7.^a ed. Pearson, 2015.

BAYER, R.; McCREIGHT, E. — *Organization and Maintenance of Large Ordered Indexes*. Acta Informatica, vol. 1, n.º 3, pp. 173-189, 1972.